Lab Exercise 5.1 – Bank Account Management using OOP and Exception Handling

Question:
Develop a Python program to implement a `BankAccount` class for managing a customer's bank account.

The class should contain:

- `__init__(self, account_no, holder_name, balance)` to initialize account details.
- `deposit(amount)` to deposit money.
- `withdraw(amount)` to withdraw money.
- `get_balance()` to display the current balance.
- `__str__()` to display the account details in a readable format.
- Use **encapsulation** by making the balance a private attribute.
- Raise a suitable exception when the withdrawal amount exceeds the available balance.
- Handle invalid input using `try-except`.
- Demonstrate the program using at least two account objects.

Concepts: Class/Object, constructor, encapsulation, private attributes, methods, `__str__`, `raise`, custom exception handling.

Lab Exercise 5.2 – Employee Payroll System using Inheritance and Polymorphism

Question:
Develop a Python program to implement an employee payroll system using **inheritance and polymorphism**.

Create a base class `Employee` with:

- Employee ID
- Employee name
- Basic salary
- A method `calculate_salary()`

Create the following derived classes:

- `PermanentEmployee`
- `ContractEmployee`
- `Intern`

Each derived class should **override** `calculate_salary()` and calculate the salary differently according to its employee type.

Requirements:

- Use `super()` to initialize the common employee attributes.
- Create objects for different employee types.
- Store the objects in a single list.
- Use a loop to call `calculate_salary()` for every object and display the result.
- Demonstrate how the **same method call produces different behaviour** for different objects.

Concepts: Inheritance, method overriding, `super()`, polymorphism, objects and lists of objects.

Lab Exercise 5.3 – Safe Student Marks Processing System

Question:
Develop a Python program to implement a `Student` class that accepts and processes student marks safely.

The program should:

- Create a `Student` class with student name, register number and marks.
- Accept marks for **5 subjects** from the user.
- Calculate total, average and grade.
- Use `try-except` to handle:
    - Non-numeric marks (`ValueError`)
    - Marks outside the valid range of 0–100 (`ValueError` or custom exception)
    - Any other unexpected input errors.
- Create a custom exception named `InvalidMarksError`.
- Use `raise` whenever an invalid mark is entered.
- Use `else` to display the result when all inputs are valid.
- Use `finally` to display `"Marks processing completed."`
- Display the student details using `__str__()`.

Concepts: Classes, objects, methods, custom exceptions, `raise`, multiple exceptions, `else`, `finally`, and `__str__`.